CSE 151B/251B Milestone Report

Varick Reynaldo
vreynaldo@ucsd.edu

Abstract

The Kaggle Math Reasoning Competition challenges participants to improve a large language model (Qwen3-4B) to its limits on mathematical reasoning using only model-intrinsic methods at inference. To be specific, we will be experimenting with prompt engineering, sampling parameters, and reinforcement learning.**Insert results, conclusion, and rank**

1 Introduction

For this competition, we will be optimizing the performance of a large language model on mathematical reasoning tasks. The model is trained on free-form and multiple choice math reasoning question and answers. Unlike a classification problem, the model needs to reason step by step to reach the answer. For example, "Find the sum of the first 325 positive even whole number", the model must first understand that this is a free form question, identify the first 325 positive even whole numbers and find the sum.

It is important to maximize a large language model's ability to perform mathematical reasoning, because math is an essential skill to perform many tasks ranging from basic to advanced. Model's which are adept at mathematical reasoning will be able to reliably answer any doubts a student may have after learning a new concept, such as gradient descent, taught in class. If it has poor mathematical reasoning skills, it could give incorrect advice/answers and confuse students. Unlike a simple google search, math reasoning models provide a quick and accesible form of help tailored to one's needs.

It is often difficult to understand how or why a model is able to arrive at a specific decision. This leads to challenges in debugging and optimizing. More specifically, a model could produce valid reasoning, but contain arithmetic errors. Prompt engineering could improve or degrade the accuracy, depending on the prompt. Furthermore, even after the model the model arrives at the correct answer, it could still marked wrong if the final output isn't in the correct format.

The starter code provides the pipeline to run the baseline. It loads the math dataset, builds the prompt, loads the model, generates the response, and evaluates the performance. It's limitations are:

- Sub-optimal prompts
- Sub-optimal sampling parameters
- Responses generated are not in the correct format
- Dataset is large and noisy

All these factors lead to a baseline accuracy of 0%. Our current solution addresses these issues by improving the prompt, experimenting with sampling parameters, and reinforcement learning.

Contribution: This is a team of one.

Preprint. Under review.

2 Related Work (Optional)[+1 pt]

Describe related papers and contextualize your own work with respect to them. If useful, you can reference and discuss them in the introduction too. Example:

Related works topic 1.

Related works topic 2.

3 Methods[2 pt]

Our method uses a pretrained small language model (Qwen3-4B-Thinking-2507) to solve mathematical reasoning problems. Due to the limitations of vllm on Windows, our model will be loaded with Transformers. The problems will be taken from a public latex dataset stored in a .jsonl file. There are three question types:

- Free-form (single): Produce a single numeric/symbolic solution.
- Free-form (multi): Produce multiple numeric/symbolic solutions for each [ans] placeholder.
- Multiple choice: Produce a letter (A, B, C, ...) corresponding to the correct option.

Each problem will be used to build a prompt (specific to free-form/multiple choice), and passed to the model for generation. The answers are then extracted and normalized from the generated responses. The method is organized into **(three?)** experiments: prompt engineering, sampling-parameter tuning, and reinforcement learning-based optimization. The improvement in performance over the baseline will be measured by validation accuracy, from a public hold-out validation set, and test accuracy, from a private dataset.

We can view the task as an optimization problem where the goal is to maximize the validation accuracy.

4 Experiments[4 pt]

In this section, we perform several experiments and observe any improvements/degradation in accuracy. These experiments will explore how prompt engineering, sampling parameters, and reinforcement learning will affect the model's output generation.

4.1 Baselines[1 pt]

We compare with the following baselines:

- **Baseline A: Starter-code** This baseline uses the starter code provided for the Kaggle competition. This will be the baseline for our **prompt engineering** experiments.
- **Baseline B: Prompt-engineered** This baseline will be the configuration of the model with the best validation accuracy after prompt engineering. This will be the baseline for our **sampling parameter tuning** experiments.

4.2 Evaluation[0.5 pt]

We evaluate each model using a validation set. The model will generate responses for batches of problems, and the final answer will be extracted from the response using a utility function from the starter code. The validation accuracy will then be calculated after the model has gone through the whole validation set.

4.3 Implementation details[0.5 pt]

All experiments were run using GPU for both training and testing. Transformeres were used for generation, instead of VLLM due to limitations on Windows systems.

- **prompt engineering:** It took on average 72s to generate one prompt. The baseline used when performing experiments for prompt engineering is baseline A.

  A few changes to the configuration were made to allow the model to run on our systems. We tested this baseline with the full dataset to get a good understanding of the initial performance. Max new tokens was set to 256, because using a higher value would be too computationally expensive and take too long to run on our system.

Table 1: Parameters for starter baseline configuration

Parameter	Value
responses generated	1126
max new tokens	256
temperature	0.6
top p	0.95
top k	20
repetition penalty	1.0
do sample	true

After running the starter baseline, we noticed that the responses ran out of available tokens before reaching the final answer, and that the responses which produced an answer did not format it correctly. So, we wanted to solve this using prompt engineering.

We plan to increase the max tokens to allow the model to finish its reasoning, ask the model to be more concise in its reasoning, make sure to double check its answers, format the user prompt to be more descriptive, and provide examples of how it should respond to a sample question.

- **Sampling parameter tuning:** It took an average of 242s to generate one prompt. The baseline used when performing experiments for sampling parameters is baseline B.

Table 2: Parameters for prompt-engineered baseline configuration

Parameter	Value
responses generated	100
max new tokens	2048
temperature	0.6
top p	0.95
top k	20
repetition penalty	1.0
do sample	False

Due to the extended period of time it takes to generate responses, it would be inefficient to test parameters by looping through combinations of values. So, we instead tested some predefined sets of sampling parameters from the QWEN3 model cardQwen Team [1].

4.4 Results[2 pt]

Table 3: Parameters for prompt-engineered baseline configuration

Experiment	Best validation accuracy	avg. runtime per response
baseline	0%	10s
prompt engineering	3%	72s
sampling parameters	7.89%	242s

Result 1: Starter Baseline We achieved a validation accuracy of 0% with the starter baseline. Looking at sample responses, it seems that we set the max tokens too low. So, all the responses reached the token limit before reaching a final answer.

Result 2: Prompt Engineering Our experiments show that changing the prompts can significantly alter the behavior of the model. Requesting the model to think step by step and to recheck their final answers improves accuracy, but this results in much longer outputs which increases runtime and risk the possibility of running out of tokens. Asking the model to be concise mitigates this effect, but still produces quite lengthy responses. Providing the model with a list of formatting rules greatly improved the chances that the model formats the questions correctly.

Result 3: Sampling Parameters Using the sampling parameter set recommended for thinking mode provided the best results.

5 Discussion [2 pt]

We achieved a slight improvement over baseline, with the experiments performed. Albeit, the results of baseline is probably an underestimation due to the low token limit.

When looking into the results of the experiments from prompt engineering, it seems like some of the responses generated for free response matched the ground truth and were formatted correctly, but the judger still marked it as incorrect. It seems like there is a package in judger.py which is not compatible with Windows.

The main bottlenecks were system resources, time and Windows.

Moving forward, I plan to implement Group relative policy optimization.

I worked alone. I have taken a few classes related to machine learning and deep learning before, but I do not have any prior experience with LLMs.

LLM Usage

- helped brainstorm ideas on how to improve prompts

References

[1] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.